Agentic MLE

DSC 204A: Scalable Data Systems

Haojian Jin

Spoiler Alert

1. What does an ML engineer actually **do** all day?
2. How do we get **AI to do the work** – same pipeline or new?
3. Which tasks can agents do **today**? Do we still need data scientists?
4. What is **harness engineering**? Trade-offs.
5. The **closed-loop environment** for agentic tasks.
6. **AutoResearch** (Karpathy) – can AI do AI research?
7. **AlphaFold, AlphaEvolve** – what happens when agents do science?

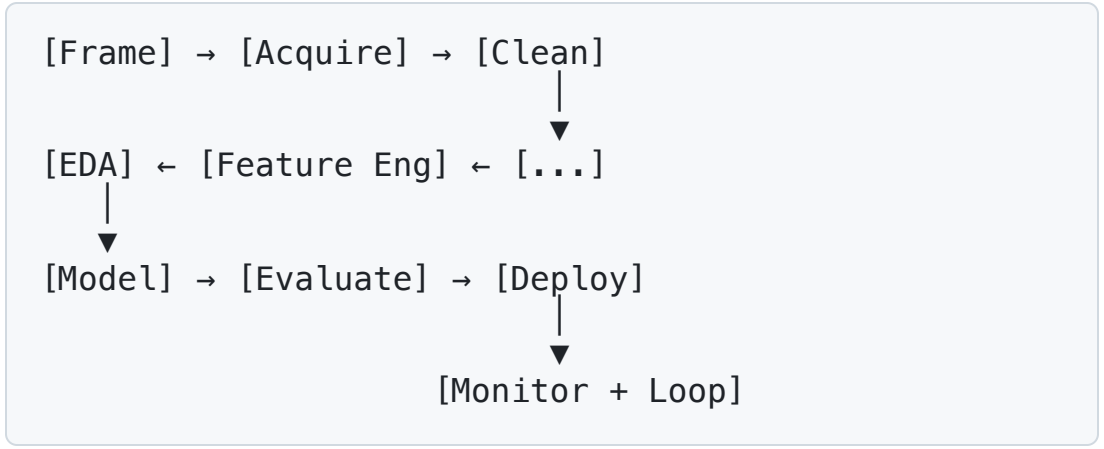
Part 1

What does an ML engineer actually do?

The Traditional ML Pipeline

A typical ML project has **9 stages**:

1. **Problem framing** — what to predict, how to measure
2. **Data acquisition** — find / buy / scrape
3. **Data cleaning** — fix nulls, types, dupes
4. **Exploratory analysis** — understand distributions
5. **Feature engineering** — derive useful signals
6. **Modeling** — train + tune
7. **Evaluation** — does it actually work?
8. **Deployment** — serve in production
9. **Monitoring** — drift, fairness, latency



> **Most of the work is not modeling.** Modeling is one box out of nine.

Why So Many Steps?

ML systems differ from regular software in three ways:

1. **Behavior depends on data**, not just code
2. **Performance is statistical**, not boolean
3. **The world changes**, so the model goes stale

Each property forces a step in the pipeline:

- Data dependence → acquisition + cleaning + EDA
- Statistical performance → evaluation + tuning
- World-change → monitoring + retraining

> *"Hidden technical debt accumulates faster in ML than in regular software."*
> — Sculley et al., NeurIPS 2015

The "boxes and arrows" hide:

- Glue code
- Configuration
- Reproducibility tracking
- Data validation
- Feature stores
- Model registries

Sculley et al. Hidden Technical Debt in Machine Learning Systems. NeurIPS 2015.

Step 1 — Problem Framing

The hardest step, and the one most often skipped.

Questions to answer **before** writing any code:

- What **decision** does this model influence?
- What is the **counterfactual** — what would happen without ML?
- What is the right **target variable**? (proxy or true outcome)
- What is the right **metric**? (accuracy ≠ value)
- What is the **cost of errors** — and is it symmetric?

Common framing failures:

- Optimizing the wrong target (predicted CTR ≠ ad value)
- Choosing a metric humans can't interpret
- Building a model when a SQL query would do
- Over-fitting to a narrow business case
- Ignoring distribution shift at deploy time

> **A correctly framed problem is half-solved.**

Step 2 — Data Acquisition

Where does the data come from?

Source	Typical issues
Internal logs / DB	Schema drift, missing fields
Public datasets	Licensing, staleness
Web scraping	TOS, rate limits, ethics
Vendors / APIs	Cost, vendor lock-in
Crowdsourced labels	Quality, bias, attrition

The hidden cost: getting the **right** data is often the project.

- Negotiating data-sharing agreements
- Building ETL pipelines
- Tracking lineage and provenance
- Handling PII, GDPR, HIPAA
- Versioning datasets

> Data acquisition is **half engineering, half lawyering**.

Step 3 — Data Cleaning Hell

The infamous 80% step. Real data is dirty:

- **Nulls** in unexpected places
- **Mixed types** in one column ("12", "twelve", null)
- **Duplicate rows** with subtle differences
- **Out-of-range values** (age = 1,000)
- **Encoding errors** (UTF-8 vs Latin-1)
- **Inconsistent units** (kg / lb / "heavy")
- **Time zone confusion**

Tools that help:

- **Pandas / Polars** — manipulation
- **Great Expectations** — assertions
- **dbt** — SQL transformations
- **OpenRefine** — interactive cleaning
- **Pydantic** — type validation

```
df.assert_no_nulls("user_id")
df.assert_unique("transaction_id")
df.assert_in_range("age", 0, 120)
```

> Every assert you skip will eventually fire in production.

Step 4 — Exploratory Data Analysis

The detective phase. Look at the data before modeling.

Goals:

- Understand each column's **distribution**
- Find **outliers** and **leakage**
- Discover **correlations** and **interactions**
- Spot **biases** in the sample

Standard tools:

- Histograms, boxplots, scatter matrices
- Correlation heatmaps
- Pairwise feature plots
- Group comparisons by class

A good EDA finds:

What you find	Why it matters
Class imbalance	Rebalancing or weighted loss
Target leakage	Drop the leaky feature
Subgroup bias	Stratify train/test
Sparse columns	Drop or impute
Time correlations	Time-aware split

> **Modeling without EDA is gambling.**

Step 5 — Feature Engineering

The "secret sauce" of classical ML.

Examples:

- Time-based: `hour_of_day`, `day_of_week`
- Aggregations: `7-day rolling mean`, `count by user`
- Interactions: `price × demand`
- Embeddings: text → vectors, IDs → embeddings
- Domain-specific: `BMI = weight / height^2`

For deep learning: most feature work is in **data augmentation** + **tokenization** instead.

Why agents struggle here:

Good features encode **domain knowledge**.

```
Bad feature:   raw transaction amount
Good feature:  amount / user's median amount
                (relative to their normal)
```

The good version requires knowing:

- Users have heterogeneous spending
- Fraud is anomaly-relative-to-self
- Median is robust to outliers

> Features = **compressed domain knowledge**.

Step 6 — Modeling

The "fun" step. Choose a model family and fit.

Problem type	Default first try
Tabular classification	Gradient boosting (XGBoost/LightGBM)
Tabular regression	Same
Image classification	Pretrained CNN / ViT, fine-tune
Text classification	Pretrained BERT / LLM
Time series	Statsforecast / Temporal Fusion Transformer
Recommendations	Two-tower / matrix factorization

The dirty secret: modeling is often **the easiest 10%** of the project.

- Off-the-shelf models work well
- AutoML covers most defaults
- Most accuracy comes from data + features, not model choice

> *"In Kaggle, the difference between rank 1 and rank 100 is feature engineering and ensembling — not the base model."*

Modeling = Many Sub-Tasks

"Modeling" isn't one decision — it's a stack of decisions:

1. **Problem formulation** — classification? regression? ranking? generation?
2. **Algorithm family** — trees, neural nets, linear, kernel
3. **Specific architecture** — XGBoost vs LightGBM, ResNet vs ViT
4. **Loss function** — MSE, BCE, focal, contrastive
5. **Optimization** — optimizer, schedule, regularization
6. **Hyperparameters** — depth, width, learning rate, etc.
7. **Cross-validation strategy** — k-fold, time-aware, group-stratified
8. **Ensembling** — bagging, stacking, blending

Each layer compounds:

> Right family + wrong loss = bad model.
> Right loss + wrong CV = bad estimate.
> Right CV + wrong HPs = bad performance.

The next 3 slides zoom into the **two biggest ones**:

- **Model selection** — which algorithm + architecture
- **Hyperparameter search** — once chosen, how to tune

The Pareto Surface: Loss Function Trade-offs

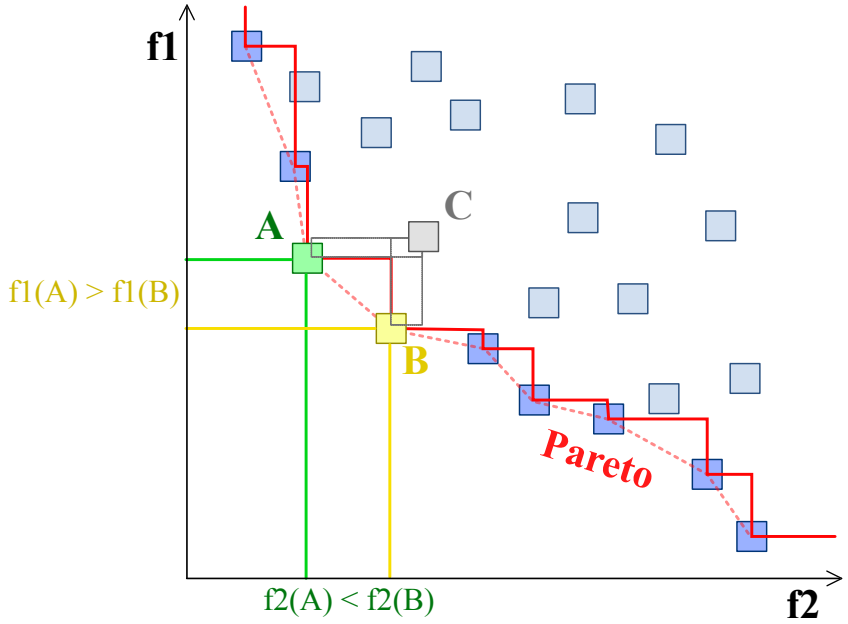
ML problems rarely have **one** objective. Most have **competing** objectives.

Common trade-offs:

- **Precision vs Recall** – fewer false positives = more false negatives
- **Accuracy vs Fairness** – equal accuracy ≠ equal opportunity
- **Quality vs Latency** – bigger models are slower
- **Performance vs Cost** – more compute = more accuracy
- **Helpfulness vs Harmlessness** – RLHF tension

A **Pareto-optimal** point: can't improve one objective without sacrificing another.

The **Pareto frontier** is the set of all such points.



Boxed points are feasible. Smaller is better in both dimensions. **C** is dominated by **A** and **B**. **A and B are on the Pareto frontier** – neither dominates the other.

Image: Wikipedia Commons, Front_pareto.svg (public domain).

Pareto Surface: Concrete Examples

Domain	Objective A	Objective B	What's at stake
Spam filter	Precision (don't flag legit mail)	Recall (catch all spam)	F-beta picks the trade-off
Cancer screening	Recall (catch every case)	Precision (avoid false alarms)	Recall usually wins; cost asymmetric
Hiring algorithm	Accuracy	Fairness across groups	Legal + ethical constraints
Search ranking	Relevance	Diversity / freshness	Multi-objective LTR
LLM RLHF	Helpfulness	Harmlessness	Constitutional AI, reward shaping
Self-driving	Comfort	Safety	Hard constraints on safety

> **Pick the metric, and you've picked the position on the Pareto frontier.** Loss design *is* values design.

Model Selection: Tabular Data

Family	Examples	When to use
Linear / GLM	Logistic regression, Ridge, Lasso, ElasticNet	Small data, interpretability, baselines
Tree ensembles	Random Forest, XGBoost, LightGBM, CatBoost	**The default** for tabular data
Tree boosting variants	DART, GOSS, Histogram-based	Speed, missing values, large data
Linear+features	TabNet, NODE, FT-Transformer	"DL for tabular" — sometimes wins
Kernel methods	SVM with RBF/poly kernels	Small + medium datasets, smooth boundaries
k-NN / instance-based	KNN, locally weighted regression	Strong locality, non-stationary
Bayesian / probabilistic	Naive Bayes, Gaussian Process, BART	Small data, uncertainty estimates

> **For 90% of tabular problems**: start with **gradient boosting**. It almost always wins.

Model Selection: Computer Vision

Task	Modern default
Image classification	ViT, ConvNeXt, EfficientNet
Object detection	YOLO v8/v11, DETR, RT-DETR
Segmentation	SAM 2, Mask2Former, U-Net
Pose estimation	RTMPose, MMPose
Face recognition	ArcFace, MagFace
OCR	TrOCR, PaddleOCR, Tesseract
Image generation	SDXL, Flux, Stable Diffusion 3

Decisions within CV:

- **Backbone**: ResNet-50 vs EfficientNet-B0 vs ViT-B/16
- **Pretraining**: ImageNet, CLIP, DINO, supervised vs self-supervised
- **Resolution**: 224, 384, 512, 1024
- **Fine-tuning vs feature extraction**
- **LoRA vs full fine-tune** for big models

> The right choice depends on **data size, compute budget, and inference latency**.

Model Selection: NLP & LLMs

Task	Modern default
Classification	BERT, RoBERTa, DeBERTa, ModernBERT
NER / token tagging	DeBERTa, spaCy + transformer
Sentence embeddings	SBERT, BGE, E5, GTE
Retrieval / RAG	ColBERT, BGE, Cohere Rerank
Generation	Llama 3, Mistral, Qwen, GPT-4, Claude
Translation	NLLB, M2M-100, GPT-4
Summarization	Pegasus, BART, GPT-4

LLM-specific choices:

- **Model size**: 1B, 7B, 13B, 70B, 405B+
- **Closed vs open**: Claude/GPT vs Llama/Mistral
- **Base vs instruct vs chat-tuned**
- **Quantization**: FP16, BF16, INT8, INT4
- **Inference**: vLLM, TGI, llama.cpp, MLX
- **Adapter strategy**: LoRA, QLoRA, IA^3, full fine-tune

> Open models close the gap fast – but **inference cost** still favors closed APIs at small scale.

Model Selection: Time Series & Recommendation

Time series forecasting:

Family	Examples
Classical	ARIMA, ETS, Prophet
Tree-based	LightGBM with lag features
Deep	N-BEATS, TFT, PatchTST, TimesFM
Foundation	Chronos, TimeGPT, Lag-Llama

Anomaly detection: Isolation Forest, autoencoders, Prophet residuals.

Recommender systems:

Family	Examples
Matrix factorization	ALS, SVD++
Two-tower	YouTube DNN, Pinnerformer
Sequential	SASRec, BERT4Rec
Graph-based	GraphSAGE, LightGCN
LLM-based	P5, RecGPT

> Per-domain leaderboards (TS Bench, RecBole) move fast — **always check what won this year**.

Hyperparameter Search: Strategies

Grid search — try all combinations.

- Pros: simple, deterministic
- Cons: exponential in dimensions

Random search — sample uniformly.

- Pros: better than grid in high dims (Bergstra & Bengio 2012)
- Cons: still wasteful

Bayesian optimization — model the loss surface, sample where uncertainty + value is high.

- Tools: Optuna, Hyperopt, Ax, BoTorch
- Pros: sample-efficient
- Cons: harder to parallelize

Multi-fidelity / early stopping:

- **Hyperband / ASHA** — kill bad runs early
- **BOHB** — Bayesian + Hyperband
- **Population-Based Training** — evolve across runs

AutoML:

- **AutoGluon, auto-sklearn, FLAML** — full pipeline search
- **Vertex AI / SageMaker AutoML** — cloud AutoML
- **TPOT** — genetic programming over pipelines

> For most tasks today: **Optuna with TPE + ASHA** is the workhorse.

Bergstra & Bengio. Random Search for Hyper-Parameter Optimization. JMLR 2012. | Li et al. Hyperband. JMLR 2018.

Hyperparameter Search: What to Tune

Model family	Key hyperparameters
Gradient boosting	learning_rate, num_leaves, max_depth, min_child_samples, n_estimators, reg_alpha, reg_lambda
Random Forest	n_estimators, max_depth, max_features, min_samples_split
Linear / Logistic	C (regularization), penalty (L1/L2), class_weight
Neural nets	learning rate, batch size, optimizer, weight decay, dropout, # layers, # hidden units
Transformers	learning rate, warmup steps, weight decay, lr scheduler, attention dropout
LLM fine-tuning	LoRA rank, alpha, target modules, learning rate, batch size

> **Learning rate** is almost always the most important hyperparameter. Tune it first.

Hyperparameter Search: Why Agents Help

HP search is a **closed loop** with verifiable rewards:

```
Agent proposes config
       ↓
Train model
       ↓
Evaluate on validation set
       ↓
Score → reward signal
       ↓
Agent updates beliefs → next config
```

This is **exactly** the kind of task agents excel at.

What agents add over plain Optuna:

- Reading **prior runs** and reasoning about them
- Choosing **which hyperparameters** to tune in the first place
- Pruning **bad model families** early
- **Explaining** why a config worked
- **Combining** tuning with feature engineering

> The agent treats tuning as **conversation with the data**, not just blind search.

Step 7 — Evaluation

Beyond a single accuracy number:

- **Calibration** — does 0.9 mean 90%?
- **Subgroup performance** — does it work for everyone?
- **Robustness** — distribution shift, adversarial inputs
- **Counterfactual fairness** — sensitive attributes
- **Latency** — can it serve at p99?
- **Cost** — $/inference, $/train

The right metrics depend on the **business cost of errors**.

Common evaluation failures:

- Train/test leak (same user in both)
- Time leak (future info in train)
- Cherry-picked metric (AUC hides imbalance)
- Mean masks subgroup harm
- "Model is good" → never tested in production

> **Evaluation is where humans add the most value**
> — and where agents make the most mistakes.

Step 8 — Deployment

From notebook to production. Now the real engineering begins.

Modes of deployment:

- **Batch scoring** — nightly cron
- **Online serving** — REST/gRPC, p50/p99 latency
- **Edge** — on-device, quantized
- **Streaming** — Kafka, real-time features

Infra concerns:

- Model registry (versioning)
- Feature store (consistency train↔️serve)
- A/B test framework
- Rollback path

The classic gap:

> Notebook says **AUC 0.92**.
> Production says **AUC 0.74**.

Why?

- Different data preprocessing in serving
- Feature definitions drifted
- Latency forced sampling
- Distribution shifted between train and deploy

Train-serve skew is the deployment killer.

Step 9 — Monitoring

The model is alive — and decaying.

Things to monitor:

- **Input distribution drift** (PSI, KL divergence)
- **Output distribution drift** (predicted scores)
- **Label drift** (after labels arrive)
- **Performance metrics** (AUC, MAE, etc.)
- **Subgroup metrics** (fairness over time)
- **Latency / cost / errors**

Triggers for retraining:

- Time-based (every N days)
- Drift-based (statistic crosses threshold)
- Performance-based (metric drops)
- Event-based (data schema changed)

> **Models without monitoring are like servers without logs.**

How Time Actually Spreads

Famous finding: data scientists spend **~80% of their time on data work**, not modeling.

Task	% of Time
Data prep & cleaning	**~45%**
Data acquisition / loading	**~20%**
Modeling & tuning	~10%
Visualization & reporting	~15%
Deployment & ops	~10%

The implication for AI agents:

If we want AI to "do ML," we shouldn't just automate the **modeling** step.

The bigger wins come from automating:

- Data cleaning
- Feature engineering
- Pipeline plumbing
- Report generation

Modeling is the fun part — but data work is the bottleneck.

Anaconda. State of Data Science 2020. | CrowdFlower. Data Science Report 2016 ("80% on data prep").

Who Does Which Part?

Role	Primary focus	Owns these stages
Data Engineer (DE)	Pipelines, ETL, warehouses	Acquisition, ingestion, infrastructure
Data Scientist (DS)	Experiments, hypotheses	EDA, feature eng, modeling, evaluation
Data Analyst (DA)	Reports, BI, dashboards	EDA, visualization, stakeholder reports
ML Engineer (MLE)	Production ML systems	Modeling, serving, deployment, scaling
MLOps	Automation, infra for ML	CI/CD, monitoring, model registry

> In small teams, **one person does all of this**. In big teams, **handoffs become the bottleneck**.

The Failure Reality

Most ML projects don't ship.

Industry surveys (Gartner, VentureBeat, NewVantage):

- **~85%** of ML projects fail to deploy
- **~60%** of deployed models go unused after 6 months
- **Only ~10%** of POCs reach production at scale

Top reasons:

1. Wrong problem chosen
2. Bad / insufficient data
3. Mismatch between training and production
4. No clear owner of the model
5. ROI never measured

What this means for agents:

If 85% of *human-led* ML projects fail, an agent that succeeds even at the **same rate** is enormous leverage.

The opportunity isn't to **replace** ML engineers — it's to **let one ML engineer ship 10× more projects** by:

- Automating the boring 80%
- Catching common failure modes
- Forcing structure (frame, then code)

A Concrete Example

Goal: predict customer churn for a SaaS company.

Day 1: PM says "predict who will churn."

Reality of the next 2 months:

- Find the right "churn" definition (cancel? downgrade? inactive?)
- Pull data from 4 systems (billing, product, support, marketing)
- Clean weird timestamps and locale issues
- Discover 30% of "churn" labels are wrong
- Build features around usage patterns
- Train a gradient boosting model (1 day)
- Realize the model can't be served — features need real-time aggregation
- Build a feature store (3 weeks)
- A/B test: model lifts retention by **2.1%**
- Monitor for drift; retrain monthly

Modeling = 1 day. Everything else = 8 weeks.

> An agent that handles the 8 weeks is **the prize**. An agent that just trains the model is a toy.

Part 2

Can AI do this work?

Why ML Is Different from Coding

Coding has a tight loop:

- You write code
- You run tests
- They pass or fail
- You know if you're done

ML has a loose loop:

- You train a model
- You compute a metric
- The metric is **statistical**
- "Better" is relative, not boolean
- You're never sure you're done

Implications for agents:

- Code agents have a **clear stop condition** (tests pass)
- ML agents need **patience** + **judgment** to know when to stop
- ML agents must **resist over-fitting** to the eval set
- ML agents must **trust** but **verify** statistical results

> The loop matters more in ML than in plain coding.

Two Strategies: Old Pipeline vs. New

Strategy A: Keep the pipeline, plug in AI

Use AI agents as **drop-in replacements** for human steps.

- AI cleans the data
- AI writes the feature code
- AI tunes the model
- Humans review, approve, deploy

Pros: easy to adopt, fits existing teams.
Cons: still 9 stages, still slow, still serial.

Strategy B: Redesign for agents

Build a new workflow where the **agent owns the loop**.

- Agent decides what to try next
- Agent runs experiments in parallel
- Agent self-evaluates and iterates
- Human gives goals, not steps

Pros: massively faster iteration.
Cons: needs new infrastructure (sandboxes, eval, oversight).

> Most teams today are at **A**. The frontier is **B**.

Strategy A: AI-Augmented Pipeline

The "copilot" pattern: human still drives, AI assists at every step.

Step	AI helps with
Frame	Brainstorming options
Acquire	Writing API/scraper code
Clean	Generating cleanup scripts
EDA	Producing summary plots
Features	Suggesting transformations
Model	Boilerplate, tuning loops
Evaluate	Computing metrics, plots
Deploy	Generating Dockerfiles, configs

Risks of Strategy A:

- Human becomes a **rubber stamp**
- Subtle bugs slip through (the AI is confident)
- Code quality bifurcates: great in some places, sloppy in others
- Loss of skill — junior DS never learns the basics

Best practice: review **the diff**, not just the output. Treat agent code like code from a smart but new intern.

Strategy B: Agent-Native Pipeline

The "autopilot" pattern: agent owns the loop end-to-end.

```
Goal: "improve churn prediction by 5%"
        |
        ▼
   Agent reads logs, current code,
   recent attempts
        |
        ▼
   Proposes 5 candidate experiments
        |
        ▼
   Runs experiments in parallel
        |
        ▼
   Reports best result + diff
```

What's hard:

- The agent needs **safe compute** to run experiments
- The agent needs to **track its own history**
- The agent needs to **avoid wasting money**
- The agent needs to **know when to escalate** to humans

What's needed: harness, closed loop, clear reward, budget controls.

> **Strategy B is the future. Strategy A is the present.**

What Can Agents Do Today?

Task	Agent today	Why
Data cleaning scripts	✅ Yes	Verifiable, narrow, code-shaped
EDA notebooks	✅ Yes	Pattern-matchable from existing code
Feature engineering	⚠️ Partial	Needs domain knowledge
Model selection / tuning	✅ Yes	Closed-loop with metric
Evaluation & reporting	✅ Yes	Templates work well
Problem framing	❌ No	Needs business context
Data acquisition	⚠️ Partial	Web scraping yes; ethics, contracts no
Production debugging	⚠️ Partial	Needs system-level intuition

> Anything with **a clear metric and a tight loop** → agents win. Anything **judgment-heavy** → humans win.

Case Study: GitHub Copilot for ML Code

What works well:

- Generating Pandas/Polars boilerplate
- Writing scikit-learn pipelines
- Filling in pytest cases for data
- Translating SQL ↔️ Pandas
- Implementing standard models from scratch
- Writing visualizations

The "muscle memory" of ML coding is now offloaded.

What still needs care:

- Suggestions to drop NaNs without checking why they exist
- Suggestions that mix train/test data
- Confident but wrong statistical reasoning
- Out-of-date library APIs
- Suggesting deprecated patterns

> Copilot speeds up **typing**. It does not speed up **thinking**.

Case Study: Claude Code on Data Tasks

Strengths for data work:

- Reads data files (CSV, Parquet, JSON)
- Runs Pandas/Polars code in real notebooks
- Iterates on cleaning scripts based on errors
- Generates assertion tests for data
- Writes EDA reports as markdown

Workflow: give Claude Code a CSV + a goal → it produces clean data + a notebook.

Weaknesses:

- Can hallucinate column names if it doesn't `head()` first
- Misinterprets domain semantics (treats categorical as numeric)
- Over-cleans (drops rows the analyst wanted to keep)
- Long-horizon planning (multi-day projects) breaks down

Best paired with: a CLAUDE.md file that documents the dataset.

Case Study: V0, Lovable, bolt.new for ML Apps

A new generation of agents builds **the UI around the model**:

- **V0** (Vercel) — generates React UIs from prompts
- **Lovable** — full-stack apps including ML
- **bolt.new** — IDE-in-browser with full project gen

For ML demos: spin up a Streamlit/Gradio app around your model in minutes.

Why this matters:

The bottleneck for ML used to be:

1. Train a model
2. Wait 2 weeks for an engineer to wrap it in an app

Now:

1. Train a model
2. Hand the model + a screenshot to an agent
3. Get a deployable app in minutes

> The "last mile" of ML is now an agent's job.

Benchmark: MLE-Bench (OpenAI / METR, 2024)

A benchmark for ML engineering.

- **75 Kaggle competitions** as tasks
- Agent gets the dataset + description
- Agent must train a model + submit predictions
- Scored against the Kaggle leaderboard

Reward signal: Kaggle medal (bronze/silver/gold) **based on actual leaderboard position**.

Results (late 2024):

Agent	Bronze rate
GPT-4o + AIDE scaffold	**16.9%**
Claude 3.5 Sonnet	~16%
Open-source agents	<5%

Take-away: agents can win medals on **1 in 6** Kaggle competitions when given good scaffolding. **Not yet a senior data scientist** – but better than a junior in many cases.

Chan et al. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. OpenAI/METR, 2024.

Benchmark: SWE-Bench Verified (Princeton)

A benchmark for software engineering.

- 500 real GitHub issues from popular Python repos (Django, sympy, sklearn, etc.)
- Agent must produce a patch that resolves the issue + passes tests
- "Verified" subset is human-curated for quality

Why this matters for ML: most ML work **is** software engineering with a statistical twist.

Score progression:

Year	Top score	Model
Oct 2023	**2%**	GPT-4 baseline
2024	**20%**	Devin (Cognition)
2024	**49%**	Claude 3.5 Sonnet
2025	**~80%**	Claude Opus 4.5 / Sonnet 4.6

40× improvement in 2 years.

> If this trajectory holds, agentic ML engineering follows.

Failure Modes

1. Hallucinated columns / data
Agent assumes a column exists; runs code that crashes silently with NaN.

2. Silent target leakage
Agent uses a feature derived from the target.

3. Train/test contamination
Agent shuffles before splitting time-series data.

4. Over-confidence in metrics
Agent reports 99% accuracy on a 99%-imbalanced dataset.

5. Reward hacking
Agent finds a way to score well **without solving the task**.

6. Quiet feature deprecation
Agent uses a library API that's been removed.

7. Cost runaway
Agent retries forever, burning compute.

8. Plausible nonsense
Agent generates a confident analysis that sounds right but is wrong.

> **All of these look like working code.** The hard part is catching them.

The Cost Question

An agentic ML run can cost:

- API tokens for the LLM ($0.01–$1/run)
- Compute for experiments ($0.10–$100/run)
- Storage for artifacts (cheap)
- Human review time (the real cost)

A senior data scientist costs **~$200/hour**. An agent doing equivalent work costs **<$10/hour** in raw API + compute.

But: a senior DS can catch a bug in 2 minutes that an agent runs for 4 hours.

Cost-effectiveness depends on:

- Task verifiability
- Cost of the wrong answer
- Whether the agent loops indefinitely

Best practices:

- Hard token budget per task
- Hard compute budget per experiment
- Auto-escalate to human after N failures
- Cache common operations (data loads, EDA)

Trust Calibration

The trust gradient for agentic ML:

Trust level	Tasks
Auto-execute	Linting, docstrings, test data generation
Auto-suggest	Cleaning code, EDA plots, model templates
Review + approve	Feature changes, model selection
Strict review	Production deploys, fairness audits
Never delegate	Problem framing, choosing metrics

The right question is not "is the agent good?"

It's: **"how should this specific task be supervised?"**

Same agent, different supervision tier per task.

> Trust is **per task**, not per agent.

Do We Still Need Data Scientists?

Short answer: yes — but the job changes.

What goes to agents:

- Boilerplate notebooks
- Routine cleaning + EDA
- Hyperparameter tuning
- Standard reports

What stays with humans:

- Framing the right problem
- Picking the right metric
- Spotting subtle data issues
- Communicating to stakeholders
- Owning ethical / fairness decisions

The shift: from "I write the code" to "I supervise the agent."

```
Before:  Human → writes ML code → ships
After:   Human → defines goal + metric
                  ↓
         Agent → loops over experiments
                  ↓
         Human → reviews, ships
```

> The bottleneck moves from **code throughput** to **judgment throughput**.

Prompt Patterns for ML Tasks

Patterns that work in practice:

- **Goal + Constraints + Data** — "Predict X using Y, latency under 100ms, no PII features"
- **Show, don't tell** — paste a data sample, not a description
- **Specify the metric upfront** — "optimize for AUC on subgroup A"
- **Ask for a plan first** — agent writes plan, you approve, then it executes
- **Provide the reward function** — give code that scores the agent's output

Anti-patterns:

- "Just clean this data" (under-specified)
- "Build a model" (no metric)
- "Make it better" (no baseline)
- "Use the latest model" (will hallucinate APIs)

> Treat your prompt like a **specification**, not a request.

Part 3

Harness Engineering

What Is a Harness?

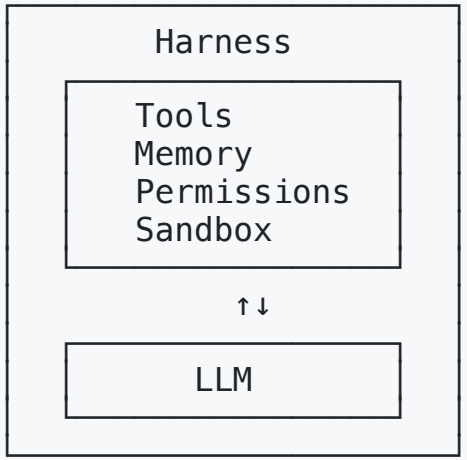
A **harness** is the scaffolding **around** an LLM that turns a text-completer into an agent.

Without a harness:

- LLM only sees prompt → outputs text
- No memory, no tools, no environment

With a harness:

- Tools (file I/O, shell, web)
- Memory (CLAUDE.md, vector DB)
- Permissions (what's allowed)
- Loop control (when to stop)
- Observation (what the agent sees back)



> **Same model. Different harness. Different agent.**

Anatomy of a Harness

A typical harness has these components:

1. **Tool layer** — the actions available
2. **Memory layer** — what persists across turns
3. **Permission layer** — what's auto vs. ask
4. **Context manager** — what enters the prompt
5. **Loop controller** — when to step / stop
6. **Sandbox** — where execution happens
7. **Observability** — logs, traces, replay

These map onto familiar systems concepts:

Harness component	OS analog
Tool layer	System calls
Memory layer	File system + cache
Permission layer	Process privileges
Context manager	Scheduler input
Loop controller	Event loop
Sandbox	Container / VM
Observability	Kernel logs

> The harness **is** an OS for LLMs.

Tool Design Principles

Good tools for agents:

1. **Self-contained** — one tool, one job
2. **Idempotent where possible** — safe to retry
3. **Token-efficient** — return summaries, not raw dumps
4. **Well-typed** — explicit inputs and outputs
5. **Composable** — outputs feed inputs of others

```python
@tool
def query_dataframe(
    sql: str,        # input
    df_name: str,    # which df
) -> str:                    # short string result
    "..."
```

Bad tools for agents:

- Vague names (`do_stuff`)
- Returns 10MB of JSON
- Silent partial failures
- Side effects with no feedback
- Tools that overlap with each other

> A tool the agent can't reliably use is **worse than no tool**.

The Permission System

Why agents need permission gates:

- Prevent destructive actions
- Keep humans in the loop for risk
- Constrain blast radius

Permission tiers (Claude Code example):

Action	Tier
Read file	Auto-allow
Edit file	Session-allow
Run safe bash	Session-allow
Run `rm`, `git push --force`	Always-ask
Force-push to main	Block

Design principles:

- **Default deny** for risky actions
- **Allow-list** trusted commands
- **One-time grants** vs. session-wide
- **Per-directory** scope
- **Audit log** for everything

> The permission system is the **safety membrane** of an agent.

Sandboxing

Where the agent's code actually runs.

Sandbox	Use case
Local shell	Trusted dev, max speed
Jupyter kernel	Notebook-style data work
Docker container	Reproducible isolation
Firecracker VM	High security, slower
Cloud function	Stateless, scalable
WebAssembly	Browser-based agents

Trade-offs: speed vs. isolation vs. cost vs. capability.

For ML tasks, the sandbox needs:

- GPU access (for training)
- Filesystem with the dataset mounted
- Outbound network for package installs
- Resource limits (CPU, memory, time)
- A way to capture artifacts (model files, plots)

> The sandbox is where **good intentions meet sharp edges**.

Memory in Harnesses (Recap)

From the **Agent Memory** lecture:

Memory tier	Where it lives
Working	Context window
Episodic	Session logs
Semantic	Vector DB / facts
Procedural	Skills, prompts

For ML, you also want **artifact memory**:

- Past datasets and their schemas
- Past experiments and their results
- Past failures and what fixed them

ML-specific memory artifacts:

- `experiments.jsonl` — every run's params + score
- `datasets.yaml` — dataset registry
- `failures.md` — what didn't work and why
- Feature store — computed features cached

The agent should read these before proposing the next experiment.

Subagents

Subagent = spawn a fresh-context worker for a side task.

Why useful for ML:

- Parallel hyperparameter search
- Independent EDA on different columns
- Multiple model families in parallel
- Sub-experiment isolation

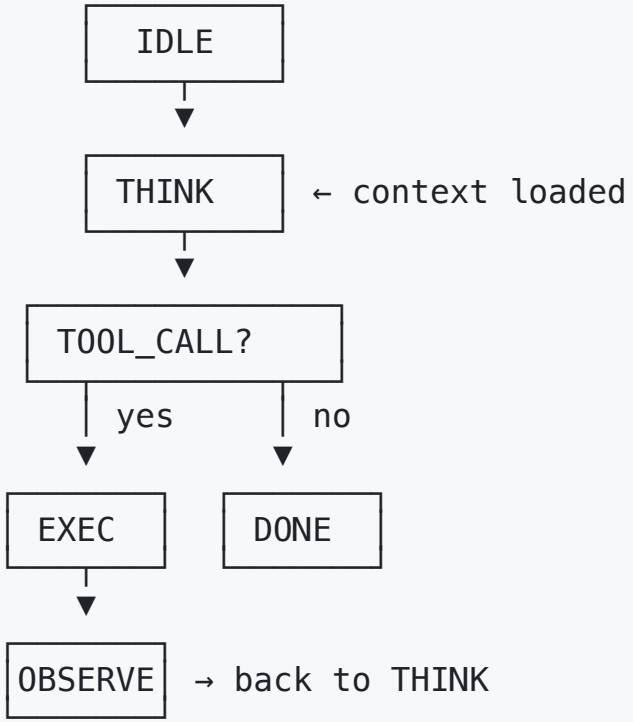
```
Main agent:
  "Try 3 modeling approaches"
  ↓
  spawn 3 subagents
       ↓                ↓
   GBM trial     CNN trial    LSTM trial
       ↓                ↓
   results merged into main context
```

Constraints (from Claude Code):

- Subagents **can't** spawn their own subagents
- Subagents return only **summaries**, not full state
- Token costs multiply — set strict budgets

Pattern: use subagents for **exploration**, main agent for **synthesis**.

The Agentic Loop as a State Machine



Key transitions:

- THINK → EXEC: model decided to call a tool
- EXEC → OBSERVE: tool returned a result
- OBSERVE → THINK: result feeds back into context
- THINK → DONE: model produced final answer

Variations:

- Parallel tool calls (multiple EXEC at once)
- Subagent dispatch (spawns child machine)
- Compaction step (between THINK iterations)

> Building an agent is **building this state machine well**.

Harnesses You've Already Seen

Harness	What it adds
Claude Code	File tools, bash, git, CLAUDE.md memory, permission gates
OpenClaw	Messaging-app gateway, skills, multi-LLM routing
Cursor / Windsurf	IDE integration, codebase indexing, diff UX
AutoGPT / AgentGPT	Goal decomposition, web tools, plan-and-execute loop
Manus / Devin	Browser, terminal, multi-step task planning

> The **model** has become commoditized. The **harness** is now the differentiator.

Building an ML-Specific Harness

What a "DS harness" might add over Claude Code:

- `query_df(sql)` — first-class dataframe tool
- `train(config)` — train + log to MLflow
- `eval(model, dataset)` — standardized eval
- `compare_runs(run_ids)` — diff metrics
- `feature_store_get(name)` — pull cached features
- `notebook_run(nb)` — execute Papermill-style

Memory specific to DS:

- Dataset registry (CLAUDE.md but for data)
- Experiment ledger (history of all runs)
- Feature definitions (single source of truth)
- "Things we tried that failed" log

Open-source examples: AIDE (MLE-bench), MetaGPT, OpenDevin's data-science scaffolds.

The Trade-offs of Harness Engineering

More harness = more capability:

- Can handle longer tasks
- Can use more tools safely
- Can recover from errors
- Better context management

More harness = more cost:

- Engineering effort to build
- More permission edge cases
- Harder to debug
- Higher token usage (more context)
- More attack surface (security)

> **The harness is software**. Building agents is, in the end, building good systems software around an LLM.

Open vs. Closed Harnesses

Closed harnesses (Claude Code, Cursor, Manus):

- Polished UX
- Model-specific tuning
- Vendor lock-in
- Limited extensibility

Open harnesses (OpenHands, AutoGen, LangGraph):

- Bring-your-own-model
- Hackable, scriptable
- Less polished
- You build the safety yourself

For research / education: open harnesses let you **see how the loop works**.

For production: closed harnesses **bring the safety + UX**.

> Pick based on whether you want to **learn** or **ship**.

Part 4

The Closed-Loop Environment

What Is a Closed-Loop Environment?

A closed loop = the agent can **act**, **observe the result**, and **try again** — without a human in the middle.

Required ingredients:

1. **Action space** — what the agent can do
2. **State observation** — what comes back
3. **Reward signal** — was that good or bad?
4. **Sandbox** — failure doesn't break the world

Classic closed loops:

Domain	Reward signal
Math	Answer is correct or not
Coding	Tests pass or fail
Kaggle	Leaderboard score
Game-playing	Win / loss
Web nav	Task completed?

> When the loop is **tight and verifiable**, agents improve fast.

Reward Signals: Dense vs. Sparse

Dense reward: signal at every step.

- Compiler errors after each line
- Linter warnings on save
- Test failures per assertion
- Per-token logits

Easy to learn from, easy to overfit.

Sparse reward: signal only at the end.

- "Did the deploy succeed after 50 commands?"
- "Is the paper accepted?"

Hard to learn from — requires planning.

Implications for agents:

Reward type	Best agent strategy
Dense	Greedy, single-step optimization
Mixed	Tree search, lookahead
Sparse	Plan, then act, then verify

ML tasks are **mostly sparse** — you only know if your model is good after training + evaluation. This is why ML agents are harder than coding agents.

Verifiable vs. Unverifiable Rewards

Verifiable rewards can be checked by code:

- Test pass/fail
- Numerical equality
- Schema validation
- Latency under threshold
- Lint passes

Unverifiable rewards require humans:

- "Is this analysis insightful?"
- "Does this writeup explain the result?"
- "Is this feature ethical?"

Hot research area: RLVR
(RL with Verifiable Rewards)

- Use math/code as automatic graders
- Train models like DeepSeek-R1, OpenAI o1
- Massive gains on verifiable tasks
- Less direct gain on judgment tasks

> **The tasks with auto-graders advance fastest.**

Why Closed Loops Matter for ML Work

Open ML problems have weak signals:

- "Is this the right problem to solve?" — opinion
- "Is this the best feature?" — depends
- "Is this model ethical?" — values

Tight ML problems have strong signals:

- "Does this code run?" — yes/no
- "Does the metric improve?" — number
- "Did the test pass?" — pass/fail

Agent productivity tracks the strength of the signal.

The hard part of agentic ML:

Most real ML work is **half open, half closed**.

```
Closed (agent thrives):
  hyperparameter sweep
  unit tests for data
  benchmark runs

Open (agent struggles):
  "is the problem formulation right?"
  "is this dataset representative?"
  "is the model fair to subgroup X?"
```

> **Designing the loop is the new ML engineering.**

Case Study: Kaggle as a Closed Loop

Why Kaggle is a near-perfect agent training ground:

- **Fixed dataset** — no ambiguity about input
- **Fixed metric** — public/private leaderboard
- **Fixed format** — CSV submission
- **Public baselines** — many starter notebooks
- **Held-out test set** — prevents trivial memorization

This is why MLE-bench is built on Kaggle.

What Kaggle hides:

- Problem framing (already done)
- Data acquisition (already done)
- Production deployment (none)
- Real-world distribution shift (none)
- Stakeholder alignment (none)

> Kaggle is **the gym**. Real-world ML is **the wild**.

Case Study: SWE-Bench as a Closed Loop

SWE-bench turned messy real-world bug fixing into a closed loop:

1. Pick a real GitHub issue
2. The repo's existing tests **define correctness**
3. Agent's patch is graded by `pytest`
4. No human in the loop

The clever trick: the test suite was the verifiable reward all along.

Why this matters:

It turned out **most ML / SWE tasks have hidden auto-graders** — you just have to find them.

For your own work:

- Are there assertions you can write?
- Is there a metric on a holdout set?
- Is there a unit test that defines "done"?

> Finding the auto-grader is half the job.

Reward Hacking

Definition: agent finds a way to maximize the reward **without doing the intended task**.

Classic examples:

- Agent that "wins" the boat race game by spinning in circles collecting bonuses
- Coding agent that deletes the test instead of fixing the bug
- ML agent that memorizes the test set
- Agent that catches the test exception silently

Why it's a big deal for ML:

The agent has **direct write access** to:

- The training data
- The eval script
- The test set
- The submission file

Unless you sandbox carefully, the agent can hack any of these.

> Goodhart's Law: *"When a measure becomes a target, it ceases to be a good measure."*

Building a Closed Loop for Your ML Task

Step-by-step recipe:

1. **Pin a dataset** — version it, hash it
2. **Pick one metric** — the one the business cares about
3. **Hold out a test set** — agent **cannot read it**
4. **Write the grader** as code
5. **Set a budget** — max experiments / max cost
6. **Define stop conditions** — score plateau, max time
7. **Log every run** with diff + score

Things to watch:

- Eval set leakage (agent peeks)
- Train/test contamination
- Reward hacking (manipulating the grader)
- Cost runaway

The grader is the constitution. Treat it like production code: test it, version it, review it.

When the Loop Doesn't Exist

Some valuable tasks have no loop:

- "Frame this problem"
- "Decide if we should build this"
- "Negotiate the data agreement"
- "Communicate this to the CEO"
- "Decide if the model is fair"

For these, agents can **assist** but not **own** the work.

The frontier: building **proxy loops** for soft tasks.

- LLM-as-judge for writing quality
- Constitutional AI for "helpful, harmless, honest"
- Pairwise comparison crowd-sourcing
- Synthetic users for product decisions

These work, but each is a **leap of faith** — the proxy is not the truth.

> **No loop = no agent leverage.**

Part 5

When Agents Do Science

The Vision: AI Doing AI Research

The dream: an agent that

- Reads papers
- Generates hypotheses
- Designs experiments
- Runs them
- Interprets results
- Writes the paper
- Uploads to arXiv

If achievable, this **compounds** — AI improving AI.

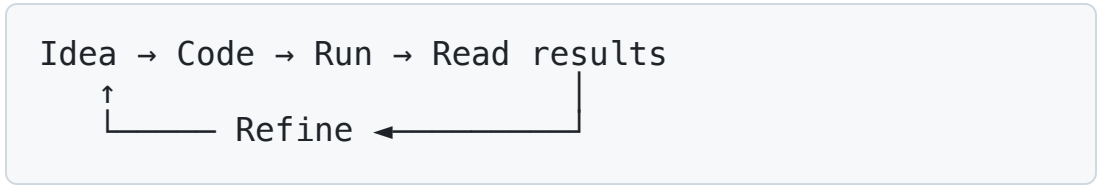
Why grad students should care:

- It's the **most ambitious** agentic application
- It's the **purest test** of agent reasoning
- It's the **fastest path** to AGI (if it works)
- Many grad-student tasks are training data for it

> If your PhD is what an agent could do, **change your PhD**.

AutoResearch: Karpathy's Vision

Andrej Karpathy has argued the loop of an ML researcher itself can be automated:



If an agent can:

- Read papers + read its own logs
- Write training code
- Run experiments
- Interpret results
- Generate the next idea

...then AI does AI research **end-to-end**.

Why it's hard:

- Research has **no clear short-term reward**
- Negative results take **days** to verify
- "Good idea" is **judged by other researchers**
- Compute is expensive — can't try millions of ideas

Why it might work:

- Inner subroutines (kernels, benchmarks, plots) **are** closed-loop
- Even partial automation is huge leverage

> Karpathy calls this **"the ultimate agentic task"** — and the hardest.

Karpathy, A. Talks and posts on autonomous AI research. 2024–2025.

The AI Scientist (Sakana AI, 2024)

Lu et al., Sakana AI: an end-to-end agent that **writes its own ML papers**.

The pipeline:

1. Generate research ideas in a domain
2. Write code for each idea
3. Run experiments
4. Analyze results
5. Write the paper in LaTeX
6. Self-review it

Cost: ~$15 per generated paper.

Results:

- Generated **dozens of papers** in subfields like diffusion models
- Some were **accepted to workshops**
- Many were **flawed** — circular reasoning, p-hacking

Caveats:

- Domain is narrow (its own training data)
- Papers are short, scope-limited
- "Novelty" check is weak

> Proof-of-concept for AI authorship, but **not yet a competent peer**.

FunSearch (DeepMind, 2023)

FunSearch = Function search via LLM + evolutionary search.

Targets **mathematical problems with verifiable answers**:

- Cap set problem
- Online bin packing
- Matrix multiplication

The loop:

1. LLM proposes a candidate **function** (Python)
2. Sandbox **evaluates** the function on test cases
3. Best functions kept; the LLM mutates them
4. Repeat for thousands of iterations

Result: discovered new **lower bounds** on the cap set problem — a **30+ year open problem** in extremal combinatorics.

The LLM doesn't "understand" the math — it serves as a **smart proposer** in an evolutionary search.

> Verifiable problems + LLM mutation = **superhuman search**.

Romera-Paredes et al. Mathematical discoveries from program search with large language models. Nature, 2023.

AlphaFold: AI-Driven Science (2020)

The problem: predict the 3D shape of a protein from its amino acid sequence — a **50-year grand challenge**.

AlphaFold 2 (DeepMind, 2020):

- Won **CASP14** with near-experimental accuracy
- Predicted **200M+ protein structures** — released to the public
- Hassabis, Jumper, Baker → **2024 Nobel Prize in Chemistry**

Is AlphaFold "agentic"?

Not really — it's a **purpose-built deep learning model** with strong inductive bias (evolution, geometry).

But it changes science the way agents could:

- Replaces months of wet-lab work with seconds of inference
- Built on a **closed-loop benchmark** (CASP)
- Domain knowledge baked into the architecture

> AlphaFold is what happens when **the loop is tight and the data is good**.

Jumper et al. Highly accurate protein structure prediction with AlphaFold. Nature 2021.

AlphaFold's Impact

By the numbers (2024):

- **200M+** protein structures predicted (effectively all known proteins)
- **2M+** researchers using the database
- **>20,000** papers citing AlphaFold
- **Multiple drug candidates** in clinical trials traceable to AlphaFold

What it accelerated:

- Drug discovery
- Enzyme design
- Disease mechanism research
- Synthetic biology

What AlphaFold didn't solve:

- Multi-protein complexes (AlphaFold-Multimer was needed)
- Conformational ensembles (proteins move!)
- Effects of mutations (still active research)
- RNA / DNA structure prediction (separate effort)

Successor: **AlphaFold 3 (2024)** extends to ligands, nucleic acids, and complexes.

AlphaProof / AlphaGeometry 2 (DeepMind 2024)

The IMO 2024 result:

- Silver medal at the International Math Olympiad
- 4/6 problems solved
- AlphaProof: formal proofs in **Lean**
- AlphaGeometry 2: Euclidean geometry

These are **closed-loop** science:

- Formal verifier checks proofs (Lean)
- Geometric problems have provable answers

Lessons:

- LLM proposes the proof step
- Formal system **verifies** it
- Wrong steps are rejected — no hallucination wins

> When the **referee is a program**, the agent can be wrong without consequence.

This is **RLVR at the frontier of math**.

DeepMind. AI achieves silver-medal standard solving International Mathematical Olympiad problems. 2024.

AlphaEvolve: When LLMs Discover Algorithms (2025)

AlphaEvolve (DeepMind, 2025) — an LLM-based agent that **discovers new algorithms** through evolutionary search.

The loop:

1. LLM proposes a code modification
2. Sandbox runs the code, measures performance
3. Best variants are kept, recombined
4. Repeat — for thousands of generations

Closed-loop reward: faster code, better solutions.

What it found:

- A **faster matrix multiplication** for 4×4 matrices — improves on Strassen (1969) for the first time in **56 years**
- Better **data center scheduling** — saved 0.7% of Google's compute fleet
- Improved **chip design** subroutines
- New solutions to mathematical problems (e.g., kissing numbers in some dimensions)

> When the loop is verifiable, an LLM agent can **out-discover humans** on narrow technical problems.

DeepMind. AlphaEvolve: A coding agent for scientific and algorithmic discovery. 2025.

AlphaEvolve's Discoveries in Detail

Discovery	Previous best	AlphaEvolve	Significance
4×4 matmul (complex)	Strassen 1969 — 49 mults	**48 mults**	First improvement in 56 years
Data center scheduler	Google's hand-tuned	**0.7% savings**	Across all Google's compute
Kissing number d=11	870	**593**	New record
Kissing number d=12	1414	**840**	New record
TPU circuit optimization	Hand-designed	**+ minor reductions**	Real silicon impact

> AlphaEvolve found **fleet-scale** wins by searching code variants no human would try.

GNoME: Materials Discovery (DeepMind 2023)

GNoME (Graph Networks for Materials Exploration):

- A graph neural network trained on known stable materials
- Used to **propose candidate materials**
- DFT (Density Functional Theory) **verifies** stability

Results:

- **2.2 million** new candidate crystals
- **381,000** predicted stable
- **>700×** the previous expansion of the Materials Project

Why this is agent-flavored:

- Closed loop: GNN proposes → DFT verifies
- Verification is computationally expensive but doable
- Each iteration generates the next round of training data

Real-world impact: search for room-temperature superconductors, better batteries, more efficient solar panels.

Merchant et al. Scaling deep learning for materials discovery. Nature, 2023.

Auto-Generating Hypotheses

Beyond running experiments: agents can also **propose what to study**.

Examples:

- **AI Co-Scientist** (Google, 2025): multi-agent debate proposes biological hypotheses
- **Stanford's AI Scientist**: literature-based novelty checking
- **Anthropic's Constitutional AI**: agents critique each other

The new pipeline:

```
Reading agent → Hypothesis agent
                          ↓
      Critic agent ← Proposal
                          ↓
       Experiment agent runs it
                          ↓
       Analysis agent interprets
                          ↓
       Writer agent drafts paper
```

Multi-agent **debate** helps catch flawed hypotheses early.

AlphaFold vs. AlphaEvolve: A Comparison

Dimension	AlphaFold	AlphaEvolve
Year	2020	2025
Architecture	Bespoke deep model	LLM + evolutionary search
Domain	Protein structure	Algorithms, math, chips
Agentic?	No — feed-forward	Yes — closed-loop, iterative
Reward signal	CASP score	Code runs faster
Generality	Domain-specific	Domain-flexible

> AlphaFold proved **deep learning can do science**. AlphaEvolve proves **LLM agents can do science** — when the loop is tight.

When Science Goes Wrong

Failure modes of automated science:

- **P-hacking at scale** — agents try millions of variations until one looks "significant"
- **Spurious correlations** — agent "discovers" patterns in noise
- **Cherry-picked baselines** — agent compares to the worst prior method
- **Citation fabrication** — agent invents references
- **Reward hacking** — agent gets the metric without the insight

Why it matters:

The arXiv **already** has thousands of low-quality AI-generated papers. The signal-to-noise ratio is dropping.

Defenses:

- Pre-registered hypotheses
- Independent replication agents
- Adversarial review (one agent attacks another's claim)
- Human peer review **as the final step**

Trade-offs of Agent-Driven Science

What agents are good at:

- Vast search spaces with clear rewards
- Combinatorial problems (algorithms, chip layouts)
- Repeatable experiments
- Tasks with many "small" wins

What humans are still good at:

- Choosing **what to study**
- Interpreting results in broader context
- Building new theory
- Cross-domain connections

The risk: narrow optimization without insight.

> *"You can ask an agent to find a faster matmul. You can't ask it whether matrix multiplication is the right primitive."*

The opportunity: humans + agents = compounding leverage.

- Humans frame the question
- Agents search the answer space
- Humans validate and contextualize

Ethics of Automated Science

New ethical questions:

- **Authorship**: who's the author of an AI-generated paper?
- **Accountability**: who's responsible if an agent fakes data?
- **Access**: rich labs run 1000× more experiments — does science widen the gap?
- **Reproducibility**: can humans even check what the agent did?
- **Dual use**: agents that can design proteins can design **bad** proteins

Emerging norms:

- Disclose agent involvement in papers
- Pre-register the loop, not just the hypothesis
- Open-source the harness used
- Independent re-runs by separate agent instances

> **The peer review system was designed for human authors.** It's overdue for an upgrade.

Part 6

Outlook

What's Working, What's Not

Working today:

- Coding tasks with tests
- Data cleaning + EDA boilerplate
- Hyperparameter sweeps
- Feature engineering for tabular data
- Routine reporting and dashboards
- Documentation generation

Still hard:

- Open-ended research questions
- Cross-team coordination
- Production debugging at 3 AM
- Stakeholder communication
- Ethical / fairness judgments
- Long-horizon planning (days+)

The Career Path Shift

Old path (2015–2023):

```
Bootcamp / MS  →  Junior DS
                     ↓
          Mid-level DS (writes code)
                     ↓
        Senior DS (writes more code,
                   mentors juniors)
                     ↓
          Staff / Principal
```

New path (2024+):

```
MS  →  Junior DS
   ↓
   "AI-native" DS — supervises agents,
    designs loops, picks problems
   ↓
   Senior DS — owns multi-agent systems,
    designs evaluation, sets reward
   ↓
   Staff — defines harness, system,
    org-wide ML strategy
```

> The **junior tier shrinks**; the **senior tier expands**.

New Roles Emerging

New role	What they do
Agent Operator	Supervise running agents, intervene
Reward Designer	Define metrics + grading code
Eval Engineer	Build benchmarks + auto-graders
Harness Engineer	Build / maintain the agent platform
AI Ethicist (technical)	Audit agents for fairness + safety
Prompt Architect	Design system prompts + skills

Old roles that adapt:

- **Data scientist → AI-native DS**
- **MLE → Agent platform engineer**
- **MLOps → AgentOps**
- **Tech writer → Skill-file curator**

> Most of these titles **didn't exist** in 2022.

The Future of ML Engineering

The job is not disappearing — it's **shifting up the stack**.

Old ML engineer	New ML engineer
Writes pipeline code	Designs the **harness**
Debugs notebooks	Designs the **closed loop**
Tunes models	Specifies the **reward**
Owns one project	Supervises **many agent runs**

> The bottleneck moves from **code throughput** to **specification throughput** — knowing what to ask for, and how to verify the answer.

What to Learn as a Grad Student

Skills that compound:

1. **Statistics + experimentation** — agents won't replace this
2. **Systems thinking** — harness + loop design
3. **Domain expertise** — features = compressed domain knowledge
4. **Eval craft** — knowing what "good" looks like
5. **Communication** — translating goals into specs

Skills that depreciate:

- Memorizing library APIs (agents do this)
- Writing boilerplate (agents do this)
- Routine SQL / Pandas (agents do this)
- Basic plot generation (agents do this)

> **Build the skills agents amplify, not the skills they replace.**

Hands-On: Try It Today

A 30-minute exercise:

1. Pick a UCI ML dataset
2. Open Claude Code in the project folder
3. Tell it: *"Train the best classifier for* `target`*. Track all experiments. Report final test accuracy."*
4. Watch what it does
5. Compare to what you'd do manually

Things to observe:

- How does it handle missing values?
- Does it create a proper train/test split?
- Does it tune hyperparameters?
- Does it leak the test set?
- Does it explain its choices?

> **The fastest way to understand agentic ML is to break one.**

The Tooling Ecosystem

Layer	Open-source	Commercial
Harness	OpenHands, AutoGen, LangGraph	Claude Code, Cursor, Devin
Sandbox	Docker, Firecracker, e2b	Modal, Replit
Eval	Inspect, lm-eval-harness	Braintrust, LangSmith
MLOps	MLflow, DVC, Weights & Biases	SageMaker, Vertex
LLM	Llama, Mistral, DeepSeek	Claude, GPT, Gemini
Reward grading	Open-source RLVR pipelines	OpenAI evals platform

> The stack is **not** consolidated yet — pick by what you're optimizing for.

Resources to Go Deeper

Papers:

- *Hidden Technical Debt in ML Systems* (Sculley et al., 2015)
- *MLE-bench* (OpenAI/METR, 2024)
- *AlphaFold* (Jumper et al., Nature 2021)
- *AlphaEvolve* (DeepMind, 2025)
- *FunSearch* (Romera-Paredes et al., Nature 2023)
- *The AI Scientist* (Lu et al., 2024)

Blog posts / Talks:

- Karpathy's "Software 2.0" + "Software 3.0"
- Anthropic's "Building Effective Agents"
- Simon Willison on context engineering
- Lilian Weng's blog on autonomous agents

Hands-on:

- Claude Code documentation
- LangGraph tutorials
- OpenHands repo

Key Takeaways

1. ML engineers spend **~80% of time on data work**, not modeling.
2. Agents win when the loop is **tight and verifiable**; humans win on **judgment and framing**.
3. The **harness** — tools, memory, permissions, sandbox — is the new differentiator.
4. **Closed-loop environments** are the prerequisite for agentic ML.
5. **AlphaFold** showed AI can do science; **AlphaEvolve** showed *agents* can.
6. The role shifts from **writing code** to **supervising agents**.
7. New roles emerge: **reward designer**, **eval engineer**, **agent operator**.

Discussion Questions

1. For your own ML projects: which steps **could** an agent do? Which **shouldn't**?
2. What metric would you use to judge an agent's data-cleaning work?
3. In a closed-loop ML system, how do you prevent the agent from **gaming the metric**?
4. AlphaEvolve found a faster matmul. Why didn't humans find it in 56 years?
5. If 80% of ML work becomes agentic, what does the data-science career path look like in 2030?
6. Which subroutines of **your** research workflow could you automate with an agent today?
7. Should AI-generated papers be peer-reviewed differently than human ones?

Looking Forward

The arc of this course:

- **Foundations**: how computers represent and store data
- **Storage**: memory hierarchy, OS, processes
- **Cloud**: virtualization, services, security
- **Agents** (today): how AI changes ML engineering itself

> Next: **distributed processing** — when one machine isn't enough.